CHAPTER 19

Rate Limits, Pricing and Quotas

In this chapter

Introduction: The Load Test That Passed . . . 3
19.1 Two Kinds of Limit, and They Are Not the Same Thing . . . 4
19.2 The Three Buckets . . . 5
19.3 The Token Bucket, and What a Burst Does to It . . . 5
19.4 Cache-Aware ITPM . . . 6
19.5 One Request, Three Different Token Counts . . . 7
19.6 Usage Tiers . . . 7
19.7 The Published Tables . . . 8
19.8 Rate Limit Groups: a Limit Is Not Per Model ID . . . 9
19.9 The 429, retry-after, and Acceleration Limits . . . 10
19.10 The Response Headers . . . 10
19.11 Spend Caps . . . 11
19.12 Workspaces, API Keys and Where Limits Live . . . 12
19.13 Reading Your Own Limits, and Attributing Your Own Cost . . . 12
19.14 Batch: a Separate Bucket and a Separate Queue . . . 13
19.15 The Buckets You Do Not Have to Share . . . 14
19.16 Service Tiers: Standard, Priority, Batch . . . 14
19.17 The Long Tail: Fast Mode, Geography and Size . . . 15
19.18 Visual Analogies . . . 15
19.19 Architecture: One Admission Controller, Not Fifty Retry Decorators . . . 16
19.20 Examples: Four Situations and the Number That Settles Them . . . 17
19.21 Code Walkthrough: A Limit Table That Refuses to Guess . . . 17
19.22 Mini Project: The Bucket Simulator, the Retry Bake-Off and the Quota Planner . . . 19
19.23 Panel A — The Three Buckets, Side by Side . . . 20
19.24 Panel B — One Request, Three Different Token Counts . . . 20
19.25 Panel C — The Plan That Passes Two Buckets and Fails the Third . . . 21
19.26 Panel D — Rate Says Build, Spend Says Scale . . . 21
19.27 Panel E — Three Clients, One Gateway, Ten Minutes . . . 22
19.28 Panel F — The First Minute Lies . . . 23

19.29 Panel G — The Same Limit, Enforced Differently 24
19.30 Evaluation and Deployment 24
19.31 Hands-on Labs 25
19.32 Lab Solutions 25
19.33 Best Practices 26
19.34 Common Mistakes 27
19.35 Performance Considerations 29
19.36 Security Considerations 29
19.37 Real Production Examples 30
19.38 Certification Notes 30
19.39 Twenty Practice Questions 31
19.40 Answers and Explanations 32
19.41 Ten Practical Exercises 33
19.42 Five Challenge Labs 33
19.43 Ten Interview Questions 34
19.44 Summary 34
19.45 Glossary 35
19.46 Further Reading 36

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Distinguish a **rate limit** from a **spend limit**, say what each one does when you reach it, and name where each is configured.
2. Describe the **three independent buckets** — RPM, ITPM and OTPM — and work out which of them binds for a given workload.
3. Explain the **token bucket algorithm** the API uses, and predict what a burst does to it.
4. Apply **cache-aware ITPM**: say exactly which token counts draw down the input bucket and which do not.
5. Read every `anthropic-ratelimit-*` **response header** and act on it *before* a 429 rather than after.
6. Handle a **429** correctly — `retry-after`, exponential backoff with jitter, and why an early retry is guaranteed to fail.
7. Locate limits correctly in the org/workspace/API-key hierarchy, and attribute cost with the **Usage and Cost API**.
8. Turn a workload description into a **usage tier**, satisfying the rate limits and the monthly spend cap separately.

NOTE

Prerequisites

Chapter 6 for tokens as the unit of everything, and for the `count_tokens` endpoint whose rate limits reappear here. **Chapter 11** for the rate card, the cache multipliers and the two-term latency model this chapter's simulator reuses. **Chapter 14** for the model registry, the per-model matrix and the rule that an unverified fact is UNKNOWN and must raise. **Chapter 16** for the request body and `wire.py`. **Chapter 17** for fixture provenance — `DOC` and `BUILT`. **Chapter 18** for effort and thinking, which is where a great many of your output tokens now come from.

Chapter 11 asked what a request costs. This chapter asks a different question: **how many of them will the platform let you make, and what stops first when you ask for more.** Prompt caching gets its full treatment in Chapter 30 and errors and retries in Chapter 29; what is here is the limit surface, and the arithmetic that turns a workload into a tier.

Introduction: The Load Test That Passed

A team is two days from launch. They have a support-triage service on Claude Sonnet 5: a 16,000-token knowledge-base prefix, a 2,000-token ticket, a 400-token reply, 200,000 tickets a day arriving over roughly ten working hours, and a peak minute about two and a half times the average one. Eight hundred requests a minute at the peak.

They run a sixty-second load test at exactly that rate. Zero 429s. **Seven hundred and fifty-six requests completed in the first minute.** They sign off.

Production degrades ninety seconds after the first burst of morning traffic, and keeps degrading. Nothing changed. The load test was honest. It was simply measuring the wrong minute.

The API rate-limits with a **token bucket**: capacity replenishes continuously up to a maximum, rather than resetting on a boundary. A bucket that has been idle is a *full* bucket, and a full input bucket holds a whole minute's allowance. The first minute of any test therefore spends the standing balance *and* the minute's income. On this workload the sustainable rate is **555.6 requests per minute** and the first minute delivered 756 — an overstatement of **36.1 per cent** that no amount of care in the test harness would have revealed, because the harness was not wrong.

That is failure one. Three more follow from it.

The client, an asyncio worker pool, retries immediately when it sees a 429. The documentation is blunt about what that buys: `retry-after` is "the number of seconds to wait until you can retry the request", and **earlier retries will fail**. Over ten minutes of the same offered load, section 19.27 measures that client making **45,045 HTTP calls to complete 6,166 requests** — 86.31 per cent of its traffic refused — while a client that reads the response headers and paces itself makes **6,361 calls, is refused zero times, and completes 195 requests more**. Seven times the traffic for slightly less work.

Then the capacity plan, which somebody did write. It checked requests per minute: 833 against a limit of 1,000, **83.3 per cent** — fine. It checked output tokens per minute: 333,333 against 400,000, **83.3 per cent** — fine. It did not check input tokens per minute, because input tokens felt like a context-window concern rather than a throughput one. Section 19.25 does check it: **750.0 per cent of the limit.** Two buckets at five-sixths full, one at seven and a half times over.

And finally the bill, which is a different constraint entirely and stops in a different way. Adding a cache breakpoint fixes the ITPM problem outright — it cuts the input tokens that draw the bucket by **5.00×**, moving the workload from a tier that does not exist on the price list into the Build tier. Except that the Build tier carries a **monthly spend cap of $1,000**, and this workload spends **$65,472 a month**. Reaching a spend cap does not throttle you. It **pauses API usage until the next calendar month.**

Four failures, one theme. Limits are not one number, they are a small family of independent constraints that fail in different ways at different timescales, and the one that binds is almost never the one you were watching. This chapter builds the arithmetic and the admission controller that make all four visible on your machine, offline, before the launch.

19.1 Two Kinds of Limit, and They Are Not the Same Thing

Before any table, one distinction. The documentation opens with it, and every subsequent confusion in this chapter comes from collapsing it.

	Rate limit	Spend limit
Unit	Requests or tokens **per minute**	US dollars **per calendar month**
What happens at the ceiling	Requests are **refused with a 429** and you may retry	API usage **pauses** until the next month
Timescale of recovery	Seconds	Weeks
Where you see it	Response headers, on every call	Console → Settings → Limits

	Rate limit	Spend limit
Who sets it	Anthropic, by usage tier; you may set workspace limits lower	Anthropic sets the tier cap; you may set your own limit below it

A rate limit is a **throttle**. A spend cap is a **switch**. Confusing the two produces the most expensive kind of incident in this chapter: an on-call engineer sees traffic stop, checks the 429 rate, finds it at zero, and looks everywhere except the billing page.

> WARNING
>
> **The asymmetry that matters**
>
> A 429 is self-healing: wait, and capacity returns. A spend cap is not: it is released by the calendar or by a support ticket. Alert on **projected monthly spend against the cap**, not only on 429 rate.

Both are set at the **organisation** level. They are not per API key. They are not, by default, per workspace. Section 19.12 covers what can be scoped downward and what cannot.

19.2 The Three Buckets

The Messages API rate limit is not one limit. It is three, measured independently, all of which must have room:

Bucket	Symbol	What it counts
Requests per minute	**RPM**	One per accepted request, regardless of size
Input tokens per minute	**ITPM**	Uncached input plus cache **writes**. Not cache reads.
Output tokens per minute	**OTPM**	Output tokens **as they are produced**

Three properties are worth stating before the numbers, because each of them surprises somebody.

The buckets are independent. Exceeding any one of them produces a 429. Having headroom in the other two does not help. A workload can be at 8 per cent of RPM and be throttled all day.

ITPM is estimated at the start of a request and reconciled during it. You are not admitted on the basis of what you actually used, because that is not knowable in advance; you are admitted on an estimate, and the estimate is corrected as the request runs.

OTPM is metered in real time, as tokens are generated. This has a consequence that catches everyone: `max_tokens` **does not factor into OTPM at all.** Setting `max_tokens` to 128,000 reserves nothing and costs nothing against the bucket. There is, in the documentation's words, "no rate limit downside to setting a higher `max_tokens` value". The teams who lower `max_tokens` to protect their rate limit are protecting nothing; the teams who lower it to bound worst-case *latency* have a real reason.

19.3 The Token Bucket, and What a Burst Does to It

The algorithm is named in the documentation: a **token bucket**. Capacity is replenished continuously rather than reset at fixed intervals.

Picture a bucket with a maximum level and a tap running into it at a constant rate. Each request removes tokens. If the level is below what you need, you are refused; you are not queued.

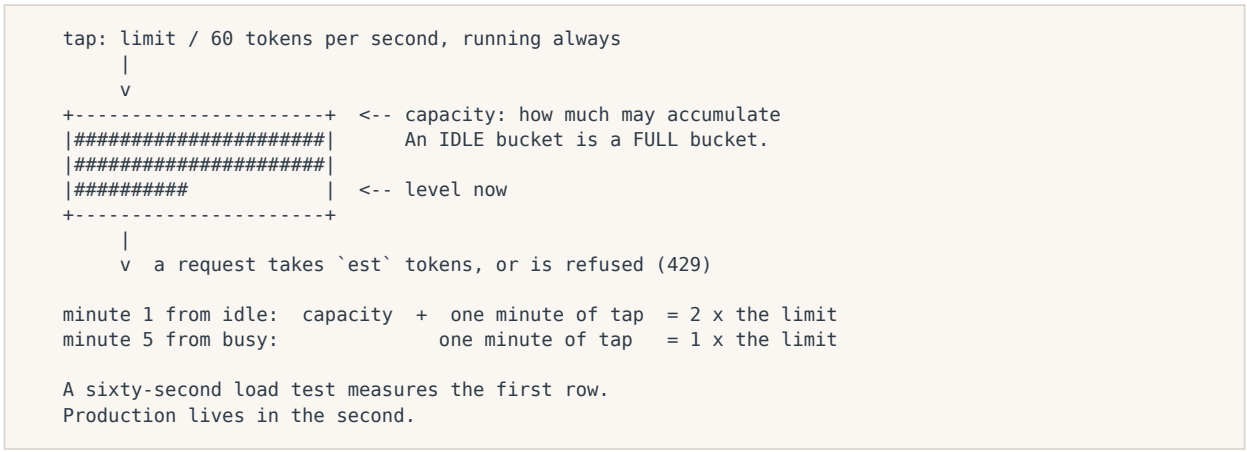


Figure 19.1 - A token bucket, and why the first minute lies

Two consequences.

Capacity is a burst allowance. How much may accumulate is a policy decision Anthropic makes and does not publish as a number. The documentation warns that a limit "might be enforced as 1 request per second" rather than as sixty per minute, and that "short bursts of requests can exceed the limit and trigger rate limit errors". So the same published RPM can behave very differently depending on the enforcement window, and you must not assume the generous interpretation.

Section 19.29 measures this. Holding everything else fixed and varying only the burst window from 60 seconds to 1 second, the naive client's completed work falls from 6,166 to 5,629 while its call count *rises* from 45,045 to 60,183. The header-paced client's falls from 6,361 to 5,814 with zero refusals throughout — because it does not need to know the burst window; the headers tell it.

> TIP
>
> **The one design rule from this section**
>
> Do not build a client whose correctness depends on your guess about the enforcement window. Build one that reads the remaining-token headers and believes them. It is then correct under any window, including one that changes without notice.

19.4 Cache-Aware ITPM

This is the single most valuable fact in the chapter, and it is the one most often missed because most API providers do not work this way.

Many providers meter one combined "tokens per minute" figure covering everything. The Claude API does not. **For most Claude models, only uncached input tokens count toward ITPM.**

Field in `usage`	Draws ITPM?
`input_tokens` (everything after the last cache breakpoint)	**Yes**
`cache_creation_input_tokens` (tokens being written to cache)	**Yes**
`cache_read_input_tokens` (tokens read from cache)	**No**

The single exception is **Claude Haiku 3.5**, on which cache reads do count. Every current model behaves as the table describes.

There is a related trap in the shape of the `usage` object itself, which the documentation spells out. `input_tokens` is **not** your total input; it is only the part after your last cache breakpoint. The total is:

```
total_input_tokens = (cache_read_input_tokens
                      + cache_creation_input_tokens
                      + input_tokens)
```

With a 200,000-token cached document and a fifty-token question you will see `input_tokens: 50`. A dashboard that plots `input_tokens` as "input" will show a beautiful, meaningless flat line.

The documentation gives the headline consequence directly: with a 2,000,000 ITPM limit and an 80 per cent cache hit rate you can process **10,000,000 total input tokens per minute** — two million uncached plus eight million cached. Prompt caching is not only a cost optimisation. **It is the cheapest rate limit increase available, and it does not require anyone's approval.**

19.5 One Request, Three Different Token Counts

Follow one request from this chapter's workload — a 16,000-token cacheable prefix, a 2,000-token tail, a 90 per cent hit rate — through the three systems that count it:

Which count	Value	Governed by
Context window occupancy	18,000	The model's context limit (Chapter 14)
ITPM bucket draw	3,600	Rate limits: hits contribute only the tail
Billed input tokens	5,440	Pricing: reads at 0.1×, writes at 1.25× (Chapter 11)

Same request. Three numbers, none of them equal, none of them wrong. The 18,000 is what has to fit. The 3,600 is what gets you throttled. The 5,440 is what you pay for. Section 19.24 derives all three from the same workload object, and this is the arithmetic that a capacity spreadsheet built on a single "tokens per request" column cannot express.

KEY TAKEAWAYS

A running thread for the rest of the book

A token has three prices: a seat in the context window, a draw on a rate limit bucket, and a line on the invoice. They are computed differently and they move independently.

19.6 Usage Tiers

Limits are defined by **usage tier**. There are four: **Start**, **Build**, **Scale** and **Custom**.

The mechanics, verified at the time of writing:

- Organisations are placed on a tier **automatically**, based on usage history and account standing, and move up over time as they use the API. Movement is not a purchase.
- **New organisations, and those with limited history, may start below the published Start figures** while account history is established. These starting limits rise automatically. If your fresh account does not match the table, that is the documented behaviour and not a fault.
- Tiers are an **organisation** property. Your tier is on Console → Settings → Limits.
- **Custom** is arranged with an account team. Its limits are not published, and code that guesses them is code that will be wrong; section 19.21 makes asking for them raise.
- On **Claude Platform on AWS**, organisations are placed on Start and **do not move between tiers automatically**; increases go through an account representative, and the self-serve *Request rate limit increase* flow is not available.

19.7 The Published Tables

Verified against `platform.claude.com/docs/en/api/rate-limits` on **30 July 2026**. These are maximum allowed usage, not guaranteed minimums.

Start tier

Model	RPM	ITPM	OTPM
Claude Fable 5	1,000	500,000	100,000
Claude Opus 5	1,000	2,000,000	400,000
Claude Opus 4.x*	1,000	2,000,000	400,000
Claude Sonnet 5	1,000	2,000,000	400,000
Claude Sonnet 4.x**	1,000	2,000,000	400,000
Claude Haiku 4.5	1,000	2,000,000	400,000
Claude Haiku 3.5 (retired except on Bedrock and Google Cloud)	1,000	100,000†	20,000

Build tier

Model	RPM	ITPM	OTPM
Claude Fable 5	2,000	1,500,000	300,000
Claude Opus 5	5,000	5,000,000	1,000,000
Claude Opus 4.x*	5,000	5,000,000	1,000,000
Claude Sonnet 5	5,000	5,000,000	1,000,000

Model	RPM	ITPM	OTPM
Claude Sonnet 4.x**	5,000	5,000,000	1,000,000
Claude Haiku 4.5	5,000	5,000,000	1,000,000
Claude Haiku 3.5	2,000	200,000†	40,000

Scale tier

Model	RPM	ITPM	OTPM
Claude Fable 5	4,000	4,000,000	800,000
Claude Opus 5	10,000	10,000,000	2,000,000
Claude Opus 4.x*	10,000	10,000,000	2,000,000
Claude Sonnet 5	10,000	10,000,000	2,000,000
Claude Sonnet 4.x**	10,000	10,000,000	2,000,000
Claude Haiku 4.5	10,000	10,000,000	2,000,000
Claude Haiku 3.5	4,000	400,000†	80,000

\ *One combined bucket across Opus 4.8, 4.7, 4.6 and 4.5.* * One combined bucket across Sonnet 4.6 and 4.5. † Counts `cache_read_input_tokens` toward ITPM.

Three things to read off these tables rather than skim past.

Claude Fable 5 has a materially tighter input bucket than its stablemates at every tier — 500,000 against 2,000,000 on Start, and 4,000,000 against 10,000,000 on Scale. Chapter 15's "a candidate is a (model, effort) pair" acquires a third dimension: the most capable model may also be the one that throttles first for a retrieval-heavy workload.

Opus 5 and Sonnet 5 each have their own bucket, while the 4.x generations share one per family. Migrating from Sonnet 4.6 to Sonnet 5 therefore does not simply move your traffic; it moves it into a bucket that was previously empty, and leaves the old one for whatever is still on it. During a phased migration you have *more* aggregate headroom than either bucket suggests.

Haiku 4.5 has the same buckets as Opus 5 and Sonnet 5. The intuition that a cheaper model gets more throughput is not supported by the table. It gets the same throughput for less money.

19.8 Rate Limit Groups: a Limit Is Not Per Model ID

The asterisks above are a rule, not a footnote. Limits are enforced against a **rate limit group**, and a group may span several model IDs and aliases. `claude-opus-4-8`, `claude-opus-4-7`, `claude-opus-4-6` and `claude-opus-4-5` all draw from one set of buckets. Traffic you moved off 4.6 last month is still competing with traffic on 4.8 today.

The authoritative mapping is not a table you maintain. It is `GET /v1/organizations/rate_limits`, which returns, per group, a `group_type`, a `models` list containing **every ID and alias** that counts against it, and

the configured `limits`. Group types are `model_group`, `batch`, `token_count`, `files`, `skills` and `web_search` — which incidentally tells you which resources have separate buckets at all.

> WARNING
>
> **The migration trap**
>
> "We moved to a newer model so our old traffic no longer matters" is true of *cost* and false of *rate limits*, unless the new model is in a different group. Check the group, not the model ID.

19.9 The 429, `retry-after`, and Acceleration Limits

Exceeding any bucket returns HTTP **429** with `error.type` of `rate_limit_error`, a message naming the limit you exceeded, and a `retry-after` header.

```
{"type": "error",
 "error": {"type": "rate_limit_error",
           "message": "Number of request tokens has exceeded your rate limit"},
 "request_id": "req_011CSHoEeqs5C35K2UUqR7Fy"}
```

The critical sentence about `retry-after` is this: it is "the number of seconds to wait until you can retry the request. **Earlier retries will fail.**" That is a contract, not advice. A retry sent before the deadline is not a gamble with poor odds; it is a request whose failure is already determined. Section 19.27 counts them: of the naive client's 38,879 refusals, **36,983 arrived before a deadline the server had already published**.

There is a second, less familiar source of 429s. **Acceleration limits** fire when an organisation's usage increases sharply, even within the published ceilings. The documented mitigation is to "ramp up your traffic gradually and maintain consistent usage patterns". Two practical consequences: a blue-green cutover that moves all traffic to a new model in one step is exactly the shape that trips this, and a retry storm is itself a sharp increase in usage — so a naive client can manufacture the condition it is reacting to.

Do not confuse 429 with **529** `overloaded_error`, which means the API is busy across all users rather than that you are over your limit. Both are retryable; only one is about you. Chapter 29 develops the full taxonomy.

19.10 The Response Headers

Every response — not only the failures — carries the state of your buckets. This is the chapter's most under-used feature: the information needed to avoid a 429 arrives on the request *before* it.

Header	Tells you
`retry-after`	Seconds to wait. Present on 429.
`anthropic-ratelimit-requests-limit` / `-remaining` / `-reset`	The RPM bucket. `-reset` is RFC 3339.

Header	Tells you
`anthropic-ratelimit-input-tokens-limit` / `-remaining` / `-reset`	The ITPM bucket.
`anthropic-ratelimit-output-tokens-limit` / `-remaining` / `-reset`	The OTPM bucket.
`anthropic-ratelimit-tokens-limit` / `-remaining` / `-reset`	The **most restrictive** limit currently in effect.
`anthropic-priority-input-tokens-*` / `-output-tokens-*`	Priority Tier only. Their presence is itself a signal.
`anthropic-fast-*`	Fast mode's separate limits.

Two details with teeth.

Token remaining figures are rounded to the nearest thousand. A client that treats `-remaining` as exact will occasionally believe it has up to 500 tokens it does not have. Treat it as an upper bound and keep a safety margin.

The undifferentiated `anthropic-ratelimit-tokens-*` family reports whichever limit is currently most restrictive — a workspace limit if one applies, otherwise the sum of input and output remaining. It is a convenience, not a fourth bucket, and it is the wrong header to build an admission controller on. Use the explicit `input-tokens` and `output-tokens` families.

19.11 Spend Caps

Each self-serve tier carries a **monthly spend cap**: the maximum your organisation can spend on the API in a calendar month.

Usage tier	Monthly spend cap
Start	$500 USD
Build	$1,000 USD
Scale	$200,000 USD
Custom	None; arranged with your account team

Read those figures next to the rate limit tables and something jumps out. On the **Start** tier you may spend 2,000,000 input tokens a minute on Sonnet 5. At $2 per million that is $4.00 a minute on input alone, so the $500 cap is **125 minutes of flat-out usage** — and this chapter's workload, which is nowhere near flat out, exhausts it in **15.0 minutes** of peak traffic. On Start and Build, the spend cap is very nearly always the binding constraint, and the rate limits are close to decorative.

The jump from Build ($1,000) to Scale ($200,000) is a factor of **200**. There is no intermediate rung. A serious production workload is a Scale workload almost by definition, and the planner in section 19.26 says so out loud rather than leaving it to be discovered on the eleventh of the month.

You can also set **your own** spend limit below your tier's cap, in Console → Settings → Limits. Do. A self-set limit is the only defence against a runaway loop, and Chapter 29's retry logic is exactly the kind of code that produces one.

> COMMON MISTAKE
>
> **Reading a spend cap as a budget**
>
> **Symptom.** All API traffic stops mid-month. No 429s. No errors in the model layer. **Cause.** The organisation reached its tier's monthly spend cap. Usage **pauses** until the next calendar month. **Fix.** Request a higher cap, or move up a tier. Not a code change. **Detection.** Alert on projected month-end spend against the cap, from the Cost API, at 60 per cent and 85 per cent.

19.12 Workspaces, API Keys and Where Limits Live

The hierarchy matters because three different things are commonly assumed to carry limits, and only one of them does by default.

```
Organisation  ......  tier, rate limits, monthly spend cap
      |               ALWAYS applies. Cannot be exceeded by
      |               the sum of anything below it.
      +-- Workspace A  ....  OPTIONAL lower rate + spend limits
      |        |             per limiter type (RPM / ITPM / OTPM)
      |        +-- API key 1 \  keys carry NO limits of their own.
      |        +-- API key 2  > They inherit the workspace, and are
      |                       /  the unit of ATTRIBUTION, not control.
      +-- Workspace B (default)
               |         cannot have limit overrides at all
               +-- API key 3
```

Figure 19.2 - Where a limit actually lives

The rules, verified at the time of writing:

- **API keys have no rate limits.** Issuing a second key does not double anything. Keys are how you *attribute* usage, not how you *ration* it.
- Workspace limits are **optional lower bounds**, set per limiter type. Absent an override, a workspace inherits the organisation's limit — which is not the same as being unlimited.
- **The default workspace cannot have overrides.** Anything you need to ration must live in a named workspace.
- Organisation limits always apply, "even if Workspace limits add up to more". Over-subscription is allowed and is often the right design: give each of four teams 30,000 ITPM out of an organisation's 40,000, and unused headroom flows to whoever needs it.
- Message Batches are **scoped to a workspace**: you can see the batches created by keys in your own workspace and no others.

19.13 Reading Your Own Limits, and Attributing Your Own Cost

Two Admin APIs matter here, both requiring an **Admin API key** (`sk-ant-admin...`), which is a different credential from your Messages key and is provisioned only by organisation admins.

The Rate Limits API — `GET /v1/organizations/rate_limits` — returns the configured limits per group, and `/v1/organizations/workspaces/{id}/rate_limits` returns workspace **overrides only**, each with the `org_limit` alongside for comparison. A group absent from the workspace response has no override and inherits; it is not unlimited. Use this at gateway start-up instead of hard-coding the tables in section 19.7, which drift.

The Usage and Cost API — `/v1/organizations/usage_report/messages` and `/v1/organizations/cost_report` — is the attribution layer. Usage buckets at `1m`, `1h` or `1d` (maximum 1,440, 168 and 31 buckets respectively) and can be grouped or filtered by API key, workspace, model, service tier, context window, `inference_geo` and, in beta, `speed`. Cost is daily-only and in USD.

Four details that save a reconciliation meeting:

- Data appears **within about five minutes**; poll no more than once a minute for sustained use.
- **Workbench usage has a** `null` `api_key_id`, because it is not made with a key.
- **The default workspace has a** `null` `workspace_id`**.** Null is not missing data; it is the default workspace.
- **Priority Tier costs are not in the cost endpoint at all** — it bills differently. Track Priority usage through the *usage* endpoint, filtered by `service_tier`.

Chapter 14 introduced Console → Usage → Export as the way to audit real model usage. This is the same data, programmatically, and it is what you build a chargeback on.

19.14 Batch: a Separate Bucket and a Separate Queue

The Message Batches API has **its own rate limits, shared across all models**. Batch traffic does not draw your Messages buckets and your Messages traffic does not draw Batch's.

Usage tier	Batch RPM	Max requests in the processing queue	Max requests per batch
Start	1,000	200,000	100,000
Build	2,000	300,000	100,000
Scale	4,000	500,000	100,000

The RPM figure governs calls to the Batch **endpoints**. The queue figure governs **individual batched requests awaiting the model** — a genuinely different unit, and the one that binds at volume. This chapter's workload is 4,400,000 requests a month; as batches of 100,000 that is **44 batches**, and a Scale queue holds five of them at once.

The rest of the envelope, verified at the time of writing: a batch is capped at **100,000 requests or 256 MB**, whichever comes first; most complete within an hour and **all expire at 24 hours**; results are downloadable for **29 days from creation**; batches are workspace-scoped; `stream`, `speed`, Threads parameters, `cache_hint`/`context_hint`, `max_tokens: 0` and research-preview mode are rejected inside a batch. Pricing is **50 per cent of standard**, which stacks with prompt caching — though cache hits are best-effort in a batch, with observed hit rates from 30 to 98 per cent, so the 1-hour cache duration is the sensible pairing.

Two behaviours specific to batch worth carrying into Part 6. The batch worker runs the same server-side agentic loop but takes **more iterations per turn** before returning `pause_turn`, because there is no open connection to hold. And it **throttles** `web_search` **per organisation** so a large batch cannot exhaust your search allowance, retrying throttled calls itself.

> TIP
>
> **Batch as a rate limit strategy**
>
> Because the buckets are separate, moving deferrable work to Batch does not merely make it cheaper — it *frees synchronous capacity* for work that cannot wait. On this chapter's workload, batching the overnight backlog halves the bill to $32,736 and removes its ITPM draw from the daytime peak entirely.

19.15 The Buckets You Do Not Have to Share

Several resources are rate-limited independently of the Messages API. Knowing this changes designs.

Resource	Limit	Note
Token counting (`/v1/messages/count_tokens`)	Start 2,000 / Build 4,000 / Scale 8,000 RPM	**Free**, and explicitly independent: "usage of one does not count against the limits of the other".
Files API	~100 requests per minute during beta	All file operations are free; content is billed as ordinary input when used.
Managed Agents	300 RPM create, 1,200 RPM read	Per organisation, separate from Messages.
Batch	See section 19.14	Separate RPM *and* a queue depth.
Web search	Its own `web_search` group	Throttled per organisation inside batch workers.

The token-counting limit is the one to exploit. It is free, it is independent, and its RPM is **twice to eight times** the Messages RPM at every tier. That means you can afford to count tokens for every request before sending it — which is exactly what a serious admission controller needs, because the ITPM bucket is charged on an estimate and your own estimate is otherwise a guess. Chapter 6 built the counter; this chapter finally explains why it has its own budget.

19.16 Service Tiers: Standard, Priority, Batch

"Service tier" is a different axis from "usage tier", and the collision of names is a reliable exam trap. Usage tier is *how much* you may use. Service tier is *how your request is scheduled*.

There are three: **Standard** (the default, best-effort), **Priority** and **Batch**.

Priority Tier capacity commitments are no longer available for purchase. Organisations with an existing commitment continue through their contract end date. It remains examinable and, if you inherit a system, operable, so the mechanics are worth stating: a commitment is a number of input tokens per minute, a number of output tokens per minute, a duration of 1, 3, 6 or 12 months and a specific model

version, targeting **99.5 per cent uptime**. Requests beyond the commitment fall back to standard automatically.

Priority capacity burns down at rates that mirror pricing rather than raw token counts — cache reads at **0.1**, 5-minute cache writes at **1.25**, 1-hour writes at **2.00**, and `inference_geo: "us"` on Claude 4.6 and later at **1.1** per token, input and output alike. Note the difference from ITPM: a cache read costs *nothing* against your rate limit but *does* draw Priority capacity, at a tenth. And critically, **a Priority request draws from Priority capacity and the ordinary rate limits**; if it would exceed the rate limits it is declined regardless.

`service_tier` accepts `"auto"` (the default — use Priority if available, else standard) or `"standard_only"`. The response's `usage.service_tier` tells you which you got. Priority is supported on all available models **except Mythos 5, Mythos Preview, Opus 5 and Sonnet 5** — that is, not on the newest models, which is worth knowing before planning a migration around it.

19.17 The Long Tail: Fast Mode, Geography and Size

Four smaller rules that each cause one specific incident.

Fast mode has its own limits. `speed: "fast"` on Opus 5 or Opus 4.8 draws **dedicated** buckets, separate from standard Opus limits, reported through `anthropic-fast-*` headers. It is a research preview, is unavailable on Claude Platform on AWS, cannot be used inside a batch, and behaves differently on older Opus: 4.7 **errors**, while 4.6 **silently runs at standard speed**. A silent no-op is the worse of the two.

Data residency does not buy capacity. `inference_geo: "us"` and `inference_geo: "global"` currently **draw from the same rate limit pool**. Splitting traffic by geography for compliance is right; expecting it to double throughput is wrong. It also costs 1.1× on price and 1.1× against Priority capacity.

Request size is a separate ceiling with its own error. Messages and Token Counting cap at **32 MB**, Batch at **256 MB**, Files at **500 MB**, and exceeding them is a **413** `request_too_large` — on the direct API, returned by Cloudflare before the request reaches Anthropic at all. Chapter 17's Files API result stands: uploading a file cuts payload bytes and changes the token bill by exactly nothing. Here it also keeps you under 32 MB.

The SDKs retry for you, twice, by default, with exponential backoff, honouring `retry-after`. Every client accepts a max-retries option. If you also wrap calls in your own retry decorator, your effective retry count is the product of the two, not the sum — a five-deep custom retry around a two-deep SDK retry is fifteen attempts, and section 19.27 shows what that does to a bucket.

19.18 Visual Analogies

VISUAL ANALOGY

Three turnstiles, not one gate

A stadium with three turnstiles: one counts people, one weighs their bags on the way in, one weighs whatever they carry out. You pass only if all three have room. Being light on bags does not get you through the people counter, and an empty stadium does not mean the bag scale has been reset.

Where it breaks. A real turnstile makes you wait; these refuse you and let you leave. And the outbound scale weighs you *as you walk*, so declaring a heavy load in advance — `max_tokens` — costs nothing at all.

VISUAL ANALOGY

A rate limit is a tap; a spend cap is the stopcock

The tap regulates flow: open it too far and it splutters, ease off and it recovers within seconds. The stopcock is in the street, it is turned by the water company on the first of the month, and no amount of easing off reopens it.

Where it breaks. A real stopcock announces itself — nothing comes out of any tap in the house. A spend cap can look, from inside a single service, like a network problem, because the requests fail at a layer your dashboards may not distinguish.

VISUAL ANALOGY

Prompt caching is a season ticket for the rate limit

The barrier counts full-price passengers. A season-ticket holder walks through without incrementing the counter — the operator still gets paid, at a tenth, but the *throughput* meter does not move. That is exactly cache-aware ITPM: reads are billed at 0.1× and draw **zero** ITPM.

Where it breaks. Buying the season ticket costs more than a single journey — a cache write draws 1.25× on the bill and its **full weight** against ITPM. And on Claude Haiku 3.5 the barrier counts season-ticket holders too. One model, one exception, and it is in the table for a reason.

19.19 Architecture: One Admission Controller, Not Fifty Retry Decorators

The failure this architecture prevents is the introduction's second one: retry logic scattered across call sites, each one individually reasonable, collectively generating a storm.

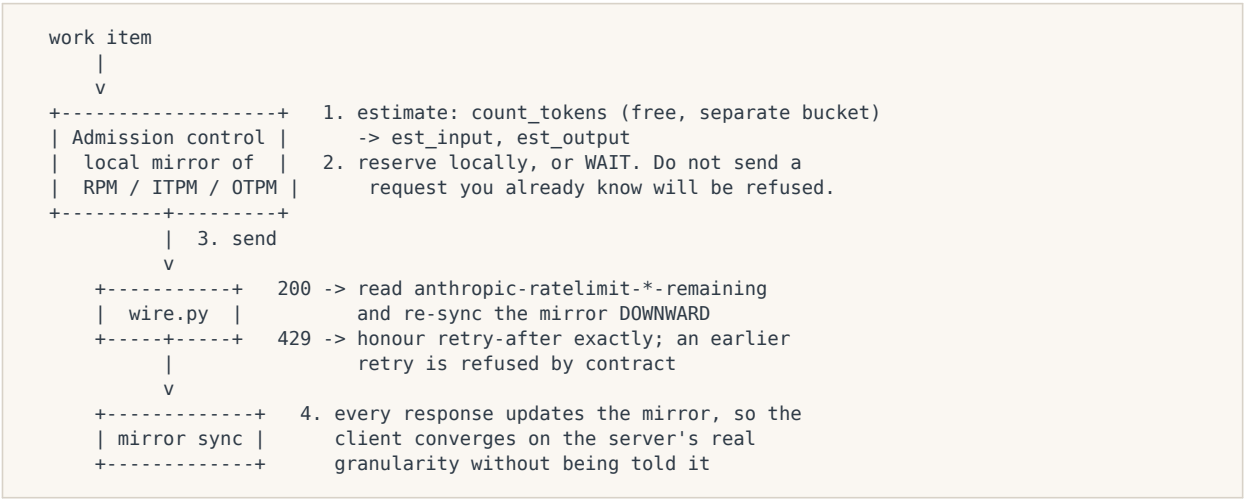


Figure 19.3 - Admission control in front of the client

Four responsibilities in one object:

1. **Estimate before sending.** The ITPM bucket is charged on an estimate; make yours, do not let it be a surprise.
2. **Reserve locally.** Concurrent workers that all read the same headroom will all spend it. A local bucket that is debited on send is what stops a hundred workers from making the same optimistic decision at once.
3. **Sync downward only.** Take the *minimum* of your reservation and the server's `-remaining`. The header is rounded to the nearest thousand and is never evidence of more headroom than you already believed you had.
4. **Own the retry policy.** One place. If the SDK also retries, know it, and count the product.

The measured payoff is in section 19.27: this design takes zero 429s across ten minutes of deliberately over-subscribed load, while the same work under immediate retry takes 38,879.

19.20 Examples: Four Situations and the Number That Settles Them

A. "Should we lower `max_tokens` to protect our rate limit?" No. OTPM is metered on tokens actually produced; `max_tokens` reserves nothing. Lower it to bound worst-case latency or to fail fast, not for throughput.

B. "We are at 8 per cent of RPM but getting 429s." ITPM. Compute uncached input per request and multiply by peak RPM. Section 19.25 shows the same shape at 83.3 per cent RPM and 750.0 per cent ITPM.

C. "Traffic stopped and there are no 429s." Check the spend cap before the code. Start is $500 and Build is $1,000; this chapter's workload passes $500 in **15.0 minutes** of peak.

D. "We halved our bill with caching — do we still need a tier increase?" Two separate answers. Caching cut the ITPM draw **5.00×** and the bill only **3.31×**, because writes are billed at 1.25× and reads at 0.1× while ITPM ignores reads entirely. The rate-limit answer and the money answer move at different speeds, and section 19.26 gives both.

19.21 Code Walkthrough: A Limit Table That Refuses to Guess

`ch19/limits.py` is the chapter's registry, built on the same discipline as `ch14/registry.py`: a figure is verified on a date or it is `UNKNOWN`, and asking for an `UNKNOWN` raises rather than returning a plausible default.

```
TIERS = ("start", "build", "scale", "custom")
SPEND_CAP = {"start": 500, "build": 1_000, "scale": 200_000,
             "custom": None}
```

Four tiers, and `custom` is present precisely so that it can refuse to answer. Its rate limits are arranged privately and are not published, so:

```
def limits_for(model_id, tier):
    if tier not in TIERS:
        raise Unverified(f"unknown usage tier {tier!r}")
    group = group_for(model_id)
    if tier == "custom":
        raise Unverified(
            "Custom tier limits are arranged with an account team and "
            "are not published. Read them from the Rate Limits API.")
    return GROUPS[group][tier]
```

A capacity planner that silently assumed Scale figures for a Custom organisation would be wrong in the direction that hurts: it would under-provision and blame the platform. Raising forces the caller to fetch the real numbers.

The grouping is explicit, because a limit is never per model ID:

```
MEMBERS = {
    "opus-5":    ("claude-opus-5",),
    "opus-4x":   ("claude-opus-4-8", "claude-opus-4-7",
                  "claude-opus-4-6", "claude-opus-4-5"),
    "sonnet-5":  ("claude-sonnet-5",),
    "sonnet-4x": ("claude-sonnet-4-6", "claude-sonnet-4-5"),
    ...
}
```

and `group_for` raises on anything it has not been told about, rather than falling through to a default group. A model released after 30 July 2026 is not "probably like Sonnet 5"; it is unknown, and the Rate Limits API knows the answer.

The heart of the module is four lines that encode the single most valuable rule in the chapter:

```
def billable_itpm(model_id, uncached, cache_write, cache_read):
    total = uncached + cache_write
    if cache_reads_count(model_id):
        total += cache_read
    return total
```

Cache **writes** count. Cache **reads** do not — except on Claude Haiku 3.5, where they do. This function is why the same eighteen-thousand-token request draws 3,600 tokens on Sonnet 5 and 18,000 on Haiku 3.5, and section 19.24 asserts both.

`ch19/bucket.py` implements the algorithm the documentation names:

```
class Bucket:
    def __init__(self, per_minute, burst_seconds=60.0):
        self.per_minute = float(per_minute)
        self.rate = self.per_minute / 60.0      # tokens per second
        self.capacity = self.rate * burst_seconds
        self.level = self.capacity
```

`rate` is the published limit. `capacity` is the *burst allowance* — how much may accumulate — and it is a separate parameter because Anthropic does not publish it and warns that a minute-scale limit may be enforced at second scale. Setting `burst_seconds=60` is the generous reading; `burst_seconds=1` is the strict one. Section 19.29 runs both and neither is assumed.

The gateway then models the three buckets exactly as documented, and the asymmetry between input and output is the part worth reading twice:

```
        wait_r = self.req.wait_for(t, 1)
        wait_i = self.inp.wait_for(t, est)
        wait_o = self.out.wait_for(t, 1)   # not max_tokens
        ...
        self.req.take(t, 1)
        self.inp.take(t, est)
        ...
    def complete(self, t, reply):
        self.out.debit(t, reply.output_tokens)
```

Input is **reserved on admission** against an estimate. Output is **debited on completion** against what was actually produced, through `debit` rather than `take` — so the output bucket may go negative, which is the honest model of a limit "evaluated in real time as output tokens are produced". Admission asks only that the output bucket is not already empty; it never reserves `max_tokens`. `test_max_tokens_reserves_nothing` sends a 128,000-token ceiling and asserts the bucket is untouched.

Finally, the contract that makes the naive client provably wrong:

```
        if t < self.hold_until:
            # Documented: "Earlier retries will fail."
            self.early += 1
            return Reply(429, self.headers(t), self.hold_until - t,
                         "hold")
```

When the gateway publishes a `retry-after`, it holds the caller to it. Every request arriving before the deadline is refused without even consulting the buckets, and counted separately as an `early` retry. This is a **modelled** contract, drawn from one sentence of documentation, and the chapter says so rather than presenting it as a recording.

19.22 Mini Project: The Bucket Simulator, the Retry Bake-Off and the Quota Planner

What it is. A capacity-planning toolkit that runs entirely offline with no API key: a three-bucket token-bucket simulator with the documented 429 contract, three client strategies replayed against it, and

a planner that turns a workload description into a usage tier.

Folder structure. It extends the code from Chapters 14 and 16 exactly as Chapters 17 and 18 do.

```
ch19/
  limits.py        the verified tier tables, groups and spend caps
  bucket.py        Bucket, Gateway, the 429 contract, DOC/BUILT fixtures
  clients.py       Naive / Backoff / Paced, and the event-driven replay
  planner.py       Workload -> demand -> tier, by rate AND by spend
  report.py        prints every figure this chapter publishes
  test_limits.py   36 tests
  registry.py      copied from ch14/ -- keep in sync
  wire.py          copied from ch16/ -- keep in sync
```

Running it.

```
cd ch19 && python3 report.py        # about four seconds
cd ch19 && python3 -m pytest -q     # 36 passed
```

The workload under study, fixed across every panel so the numbers are comparable:

Parameter	Value
Model	`claude-sonnet-5`
Requests per day	200,000, over 10 active hours
Peak factor	2.5 × the mean active minute → **833.3 rpm**
Prompt	16,000-token cacheable prefix + 2,000-token tail
Output	400 tokens
Cache hit rate	0.0 or 0.9, depending on the panel
Working days per month	22

19.23 Panel A — The Three Buckets, Side by Side

```
1. The three buckets, Claude Sonnet 5
-------------------------------------
  start  RPM   1,000  ITPM   2,000,000  OTPM   400,000  cap $500
  build  RPM   5,000  ITPM   5,000,000  OTPM 1,000,000  cap $1,000
  scale  RPM  10,000  ITPM  10,000,000  OTPM 2,000,000  cap $200,000
```

Two observations that the table makes and prose does not. **Output is always exactly one fifth of input** across all three tiers, which mirrors the pricing ratio from Chapter 11 — output is 5× the price and 1/5 the throughput, so the two constraints agree. And the spend cap does **not** scale with the buckets: Start to Build multiplies the buckets by 2.5 and the cap by 2, while Build to Scale multiplies the buckets by 2 and the cap by **200**.

19.24 Panel B — One Request, Three Different Token Counts

```
2. One request, three different token counts
-------------------------------------------
  context window       18,000 tokens
  ITPM bucket           3,600 tokens
  billed as input       5,440 tokens
```

The same request, counted by three systems that do not agree and are not supposed to.

- **18,000** is what has to fit in the context window: the prefix plus the tail, cached or not.
- **3,600** is the ITPM draw: `2,000 + 0.1 × 16,000` — the tail always, plus a cache **write** on the 10 per cent of requests that miss. Reads contribute nothing.
- **5,440** is what you pay for: `2,000 + 0.9 × 16,000 × 0.1 + 0.1 × 16,000 × 1.25` — reads at a tenth, writes at 1.25.

The ratio matters as much as the numbers. Caching divides the **rate limit** draw by 5.00 and the **bill** by only 3.31. Anyone who reasons about caching purely as a cost lever will under-estimate what it does for throughput by half as much again.

19.25 Panel C — The Plan That Passes Two Buckets and Fails the Third

```
3. Quota plan: which bucket actually binds
-----------------------------------------
  no cache  peak  833.3 rpm | start: RPM  83.3%  ITPM  750.0%  OTPM  83.3%
            tier by rate custom | by spend scale  ($176,000/mo) => CUSTOM
  cached    peak  833.3 rpm | start: RPM  83.3%  ITPM  150.0%  OTPM  83.3%
            tier by rate build  | by spend scale  ($65,472/mo) => SCALE
```

This is the introduction's third failure, measured. On the Start tier, uncached, the workload uses **83.3 per cent of RPM**, **83.3 per cent of OTPM** — and **750.0 per cent of ITPM**. Two buckets comfortable, one over by a factor of seven and a half.

It is worth being precise about why this is the *normal* shape rather than an unlucky example. Output tokens are the ones everybody counts, because they are what the model writes and what dominates latency and price. Input tokens are the ones nobody counts, because they arrive as "context" — a document, a knowledge base, a conversation history — and feel like a context-window question. But the published ratio is 5:1 input to output, and modern prompts are far more lopsided than that. This workload is 45:1. Any retrieval-augmented, long-context or agentic workload will be. **The bucket that binds is almost always ITPM, and it is almost never the one on the dashboard.**

Adding a cache breakpoint takes ITPM from 750.0 per cent to 150.0 per cent of Start, and from 300.0 to 60.0 per cent of Build. `test_caching_moves_the_required_tier_from_custom_to_build` asserts the tier change directly. One `cache_control` block is the difference between a workload that fits a self-serve tier and one that requires a conversation with sales.

19.26 Panel D — Rate Says Build, Spend Says Scale

The planner deliberately computes the two constraints separately and only then takes the higher:

```
def plan(w, price_in, price_out):
    by_rate = tier_by_rate(w)
    by_spend = tier_by_spend(w, price_in, price_out)
    needed = by_rate if ORDER.index(by_rate) >= ORDER.index(by_spend) \
        else by_spend
```

For the cached workload, `by_rate` is **build** and `by_spend` is **scale**, so the answer is **Scale**. The rate limits were never the problem; the $1,000 monthly cap on Build was. `test_but_the_spend_cap_overrules_the_rate_limit` pins all three values.

And because Sonnet 5's introductory pricing ends on 31 August 2026:

```
4. The same plan on 1 September 2026 ($3/$15)
---------------------------------------------
  no cache  $   176,000  ->  $   264,000
  cached    $    65,472  ->  $    98,208
```

The uncached plan crosses the Scale cap of $200,000 on **1 September without a single line of code changing**, and `test_september_pricing_pushes_the_cold_plan_past_the_scale_cap` asserts that its required tier becomes `custom` on that date. This is Chapter 14's $14,000-becomes-$21,000 migration finding in a different currency: a date in the price list is a change to your architecture's requirements, and it belongs in the same calendar as your deprecation dates.

Two further figures the planner prints on request. The uncached plan at $176,000 a month sits at **88.0 per cent of the Scale cap** — a 14 per cent traffic increase pauses the organisation. And at peak the uncached workload spends **$33.33 a minute**, which consumes the entire Start cap of $500 in **15.0 minutes** and the Build cap of $1,000 in **30.0 minutes**.

19.27 Panel E — Three Clients, One Gateway, Ten Minutes

Ten minutes of 800 rpm offered load, 120 workers sharing a 90-connection pool, against the Start tier's Sonnet 5 buckets. Every strategy sees the same gateway, the same seed and the same arrivals.

```
5. Retry strategy, 10 minutes of 800 rpm offered load
-----------------------------------------------------
  strategy          attempts    429s   wasted    done  first min  last min
  naive-immediate     45,045  38,879   86.31%   6,166        756       536
  backoff-jitter      12,514   6,339   50.66%   6,175        756       537
  header-paced         6,361       0    0.00%   6,361        756       558
  header-paced makes 7.08x fewer calls and completes +195 more requests
  ITPM ceiling 555.6 rpm; first minute overstates it by 36.1%
```

Read the columns in order.

Attempts. The naive client makes **45,045** HTTP calls. The paced client makes **6,361** — one per request, because it never sends a request that will be refused. That is a **7.08×** difference in traffic to the same endpoint, and of the naive client's 38,879 refusals, **36,983 arrived before a `retry-after` deadline the**

server had already published. Those are not unlucky. Their failure was determined before they were sent.

Work done. This is the part that surprises people, and it is worth stating carefully because the naive intuition is only *partly* right. Naive retrying does not halve your throughput — the bucket refills at the same rate whatever the client does, so most of the work still gets through. It completes **6,166** against the pacer's **6,361**: a loss of 195 requests, or **3.2 per cent**. The loss is real and it comes from a specific mechanism — doomed retries hold connections from a bounded pool, and a connection held by a request that cannot succeed is a connection not available to one that can.

> WARNING
>
> **A negative result, published**
>
> If your client's workers are plentiful and its connection pool is unbounded, immediate retrying costs you **almost nothing in throughput**. The 3.2 per cent above is a consequence of the modelled 90-connection pool; with 200 connections the three strategies converge. The honest summary is therefore *not* "naive retries are slow". It is: **naive retries generate seven times the traffic, produce a self-inflicted usage spike of exactly the shape that triggers acceleration limits, and buy you nothing.** That is a sufficient reason to fix it without overstating the case.

Backoff. Exponential backoff with full jitter, floored at `retry-after` — what the official SDKs do — cuts the waste from 86.31 to 50.66 per cent and completes essentially the same work. It is a large improvement for four lines of code, and it is still reacting rather than planning: half its calls are still refusals, because backoff answers "how long do I wait after being told no" and never asks "should I have asked".

Pacing. The admission controller answers the second question, and takes **zero** 429s across ten minutes of deliberately over-subscribed load. The mechanism is in `clients.py`:

```
    def commit(self, t, est):
        # Reserve locally, so concurrent workers cannot all read the
        # same headroom and all spend it.
        self.inp.take(t, est)
        self.req.take(t, 1)
    def observe(self, t, reply):
        # Trust the smaller of the local reservation and the server's
        # figure. The header is rounded to the nearest thousand, so it
        # is never evidence of more headroom than you thought.
        self.inp.level = min(self.inp.level, float(ri))
```

Two lines carry the whole design. `commit` **reserves on send**, which is what stops 120 workers from all reading the same headroom and all deciding to spend it. `observe` syncs **downward only**, taking the minimum of the local reservation and the server's `-remaining` — because that header is rounded to the nearest thousand and can therefore only ever be optimistic. A pacer that trusted it upward would drift into exactly the refusals it exists to prevent.

19.28 Panel F — The First Minute Lies

The last two columns of Panel E are the introduction's first failure, isolated. Every strategy completes **756** requests in its first minute and between **536 and 558** in its last. The sustainable ceiling is arithmetic:

```
ITPM limit 2,000,000 / 3,600 tokens per request = 555.6 requests per minute
```

The first minute delivered 756 — an overstatement of **36.1 per cent** — because a bucket that has been idle is a bucket that is full, and the first minute spends the standing balance as well as the minute's income. The pacer's last minute, 558, lands on the ceiling to within half a per cent, which is the tightest available confirmation that the model is right.

`test_the_first_minute_overstates_the_sustainable_rate` asserts both halves for all three strategies: first minute above 1.3× the ceiling, last minute at or below it.

BEST PRACTICE

How to load-test a rate-limited API

Run for at least five times your longest plausible burst window, and report the **last** minute, not the mean and never the first. If your test harness reports a single throughput number for the whole run, it is reporting the burst allowance amortised over the run length, which is a number with no operational meaning.

19.29 Panel G — The Same Limit, Enforced Differently

Nothing changes here except the burst window: how much of the minute's allowance the bucket may hold at once. The published limits are identical in all nine rows.

```
6. Burst granularity: the same limit, enforced differently
------------------------------------------------------------
    burst  strategy          attempts    429s    done  first min
    60.0s  naive-immediate     45,045  38,879   6,166        756
    60.0s  backoff-jitter      12,514   6,339   6,175        756
    60.0s  header-paced         6,361       0   6,361        756
    10.0s  naive-immediate     57,548  51,833   5,715        658
    10.0s  backoff-jitter      14,071   8,347   5,724        702
    10.0s  header-paced         5,901       0   5,901        756
     1.0s  naive-immediate     60,183  54,554   5,629        557
     1.0s  backoff-jitter      14,404   8,607   5,797        639
     1.0s  header-paced         5,814       0   5,814        720
```

As the enforcement window tightens from a minute to a second, the naive client's completed work falls **8.7 per cent** while its call count *rises* **33.6 per cent** — it is punished twice for the same mistake. The paced client falls **8.6 per cent** and takes **zero** refusals at every granularity.

That last row is the point of the panel. **The paced client was never told the burst window.** It is constructed with the generous 60-second assumption and corrects itself within one round trip, because `observe` syncs downward from a header that reflects whatever the server is actually doing. A client built on an assumption about enforcement granularity is a client that breaks when the assumption changes; a client built on the headers is not.

19.30 Evaluation and Deployment

The toolkit ships as a planning artefact and a runtime component, and they are used differently.

As a planning artefact, `planner.py` belongs in the repository next to your architecture decision records. Give it the real workload — measured from the Usage API, not estimated — and re-run it whenever the prompt shape, the model, the traffic or the price list changes. Its output is a tier and a binding bucket, which is precisely the content of a rate limit increase request. The documentation asks for exactly this when you request one on Claude Platform on AWS: the models, your peak input and output tokens per minute for each, and what share of your input is cached.

As a runtime component, the `Paced` strategy is the shape to copy, not the code. In production, seed the local mirror from the **Rate Limits API** at start-up rather than from the tables in section 19.7, which drift; make the mirror shared across processes if your workers are, because per-process mirrors over-subscribe by the number of processes; and keep the safety margin explicit, because `-remaining` is rounded to the nearest thousand.

Verification. 36 tests, all offline, all deterministic. The figures are asserted, not described: `test_the_plan_passes_two_buckets_and_fails_the_third` pins 83.3 / 83.3 / 750.0 per cent, `test_the_monthly_figures` pins $176,000 and $65,472, and `test_header_pacing_never_gets_a_429` pins the zero. If a future documentation change moves a number, a test fails and the chapter's claim fails with it, which is the intended behaviour.

19.31 Hands-on Labs

Lab 1 — Find your binding bucket. Take a real prompt from your own system. Compute its context tokens, its ITPM draw and its billed input tokens as three separate numbers, then multiply by your peak RPM and divide by the Start tier limits. **Success:** you can name which of the three buckets binds, and the ratio by which it does.

Lab 2 — Prove that `max_tokens` is free. Extend `test_max_tokens_reserves_nothing` to sweep `max_tokens` from 1,000 to 128,000 and assert the OTPM bucket is identical at every value. **Success:** the test passes for all values, and you can explain to a colleague why lowering `max_tokens` does not raise throughput.

Lab 3 — Break the pacer, then fix it. Delete the `commit` method from `Paced` so it reserves nothing and relies only on `observe`. Re-run Panel E. **Success:** refusals rise from zero, you can explain why concurrency is the cause, and restoring `commit` returns them to zero.

Lab 4 — Cross the cap. Using `planner.py`, find the exact daily request volume at which the cached workload crosses the Scale spend cap of $200,000, under both the introductory and the September price. **Success:** two numbers, and a sentence explaining why they differ by a factor other than 1.5.

Lab 5 — Split the workload. Model moving 60 per cent of the day's requests to the Batch API. Recompute the peak-minute ITPM draw on the synchronous path and the monthly spend. **Success:** you can state the new required tier by rate and by spend, and say how many batches of 100,000 the monthly volume needs.

19.32 Lab Solutions

Lab 1. The three counts are `prefix + tail`, `billable_itpm(model, tail, miss_rate × prefix, hit_rate × prefix)` and the Chapter 11 multipliers. For the chapter workload they are 18,000, 3,600 and 5,440. Divide `833.3 × 3,600 = 3,000,000` by the Start ITPM of 2,000,000 to get 150.0 per cent, against 83.3 per cent on both other buckets. ITPM binds by 1.80× relative to the next constraint.

Lab 2. OTPM is metered on tokens produced. `Gateway.send` calls `self.out.wait_for(t, 1)` — it asks only that the output bucket is not already empty — and never `takes` against it; `complete` debits the real figure afterwards. The sweep therefore passes trivially, which is the point: the absence of code is the evidence.

Lab 3. Without `commit`, all workers that become ready in the same instant read the same mirror level, all conclude there is room, and all send. The mirror is only corrected when the first reply arrives, by which time the rest are in flight. Refusals appear in bursts equal to the number of simultaneously ready workers. `commit` makes the reservation atomic with the decision, which is the standard fix for any optimistic-concurrency over-subscription.

Lab 4. At $2/$10 the cached workload costs $65,472 a month for 200,000 requests a day, so the cap is crossed at `200,000 × 200,000 / 65,472 ≈ 611,000` requests a day. At $3/$15 it costs $98,208, giving `≈ 407,000`. The ratio is exactly 1.5 because both input and output prices rose by the same factor — had only one changed, the ratio would sit between the two.

Lab 5. The synchronous peak falls to 40 per cent of 833.3 = 333.3 rpm, so the ITPM draw falls to 1,200,000 — **60.0 per cent of the Start tier**, and the workload now fits Start on rate limits alone. Spend falls to `0.4 × 65,472 + 0.6 × 65,472 × 0.5 = $45,830`, still far above Build's $1,000 cap, so **Scale is still required, and still for the spend cap rather than the rate limits.** The monthly batch volume of `4,400,000 × 0.6 = 2,640,000` requests is 27 batches of 100,000, comfortably inside a Scale queue of 500,000.

19.33 Best Practices

BEST PRACTICE

Cache before you ask for a tier increase

A cache breakpoint on a stable prefix is a rate limit increase you can deploy this afternoon, and on this chapter's workload it is worth **5.00×** on the ITPM bucket. Request the increase afterwards, with the cache rate in the request.

BEST PRACTICE

Read the headers on every response, not only on failures

`anthropic-ratelimit-input-tokens-remaining` on a 200 is the cheapest telemetry in the platform. Export it as a gauge and alert when it drops below a multiple of your largest single request.

BEST PRACTICE

Estimate with `count_tokens`, which has its own budget

It is free, and its RPM is two to eight times your Messages RPM at every tier. There is no reason for an admission controller to guess its input size.

BEST PRACTICE

Set your own spend limit below the tier cap

The tier cap protects Anthropic from you. A self-set limit protects you from your own retry loop. Both are on Console → Settings → Limits.

BEST PRACTICE

Put deferrable work on Batch for the capacity, not only the discount

Batch has separate buckets and a separate queue. Moving work there frees synchronous ITPM for work that cannot wait, and halves its price on the way.

BEST PRACTICE

Read your limits from the API at start-up

`GET /v1/organizations/rate_limits` is the source of truth. Tables in documentation and in books — including section 19.7 — drift. Reconcile, do not hard-code.

BEST PRACTICE

Ramp, do not step

Acceleration limits fire on sharp increases even within your ceilings. Phase a cutover over minutes, and treat any deployment that moves all traffic at once as a rate limit event in its own right.

19.34 Common Mistakes

COMMON MISTAKE

Watching output tokens and being throttled on input

Symptom. 429s at a small fraction of RPM, with an output-token dashboard showing plenty of headroom. **Cause.** ITPM is the binding bucket for any retrieval-heavy prompt. The chapter workload runs at 83.3 per cent RPM and 750.0 per cent ITPM. **Fix.** Compute uncached input per request × peak RPM. Add a cache breakpoint. **Detection.** Plot `anthropic-ratelimit-input-tokens-remaining`, not just token counts.

COMMON MISTAKE

Retrying before `retry-after`

Symptom. A 429 rate that rises when you retry harder. **Cause.** The documentation states that earlier retries will fail. They consume connections and inflate your usage curve. **Fix.** Honour `retry-after` as a floor, with jitter above it. **Detection.** Count refusals that arrive before the last published deadline. Section 19.27 finds 36,983 of 38,879.

COMMON MISTAKE

Treating a spend cap as a throttle

Symptom. Traffic stops dead mid-month with no 429s and no model errors. **Cause.** The organisation reached its tier's monthly cap. Usage **pauses** until the next calendar month. **Fix.** Raise the cap or the tier; there is no client-side remedy. **Detection.** Alert on projected month-end spend from the Cost API at 60 and 85 per cent.

COMMON MISTAKE

Lowering `max_tokens` to protect the rate limit

Symptom. Truncated responses and no change in 429 rate. **Cause.** OTPM counts tokens produced. `max_tokens` reserves nothing against it. **Fix.** Lower it to bound latency or cost variance, not throughput. **Detection.** Sweep `max_tokens` against OTPM remaining; the two are uncorrelated.

COMMON MISTAKE

Issuing more API keys to get more capacity

Symptom. No change in throughput after splitting traffic across new keys. **Cause.** Rate limits are per **organisation**. Keys carry none of their own. **Fix.** Keys are for attribution. For rationing, use workspace limits; for capacity, a tier increase. **Detection.** Group the Usage API by `api_key_id` and confirm the total is unchanged.

COMMON MISTAKE

Load-testing for sixty seconds

Symptom. A clean test and a production incident ninety seconds after launch. **Cause.** An idle bucket is a full bucket. The first minute spends the standing balance and the minute's income. **Fix.** Run for five or more minutes; report the last minute. **Detection.** Section 19.28 measures the overstatement at **36.1 per cent**.

COMMON MISTAKE

Plotting `usage.input_tokens` as "input tokens"

Symptom. Input usage appears to collapse the day caching is enabled, and the graph never recovers. **Cause.** `input_tokens` counts only what follows the last cache breakpoint. **Fix.** Sum `cache_read_input_tokens + cache_creation_input_tokens + input_tokens`. **Detection.** Compare the sum against the Usage API's total for the same window.

COMMON MISTAKE

Assuming the model you migrated off no longer competes

Symptom. A migration to a newer 4.x model produces no extra headroom. **Cause.** Opus 4.8, 4.7, 4.6 and 4.5 share one rate limit group; Sonnet 4.6 and 4.5 share another. **Fix.** Check `group_type: "model_group"` in the Rate Limits API. Only Opus 5 and Sonnet 5 have their own buckets. **Detection.** `group_for()` in `limits.py`, or the `models` list in the API response.

19.35 Performance Considerations

Rate limits are a throughput constraint, so every Chapter 11 latency lever reappears here with a second motive.

Caching is a throughput lever, not only a cost lever. It accelerates prefill and it removes cache reads from ITPM entirely. On this workload it is worth 5.00× on the bucket and 3.31× on the bill; those are different numbers and the larger one is the one that keeps you serving.

Fewer output tokens still improves everything at once. Chapter 11's rule survives: output tokens dominate latency, cost 5× input, and consume the bucket with the tightest headroom relative to typical usage. Chapter 18's `effort` is now also a rate limit control, because effort governs all tokens including thinking, and thinking tokens are output tokens.

Concurrency should be sized to the binding bucket, not to the worker count. At 3,600 ITPM tokens per request the Start tier sustains 555.6 requests a minute; at 7.2 seconds per request that is 67 concurrent requests. Running 120 workers against a 67-request ceiling does not increase throughput, and under a naive retry policy it actively reduces it. Size the pool from the bucket.

Batch changes the shape of the problem, not the size. A batch is not faster; it is asynchronous, half price, and served from a different bucket. The performance argument for it is that it removes load from the constrained path.

19.36 Security Considerations

Quota is a denial-of-service surface. Rate limits are per organisation, so any tenant, feature or bug that can spend tokens can starve every other tenant. This is the security case for workspaces: a compromised or runaway component confined to a workspace with its own ITPM ceiling cannot consume the organisation's capacity. The default workspace cannot be limited, which is a strong argument for not running production traffic in it.

Admin API keys are a different blast radius. `sk-ant-admin...` keys read your usage, cost and limit configuration across the whole organisation. They should never sit in an application container; they belong in the reporting job that needs them, with their own rotation schedule.

Spend caps are a safety control, not a billing preference. A self-set limit below the tier cap is the last line of defence against a prompt-injection-driven loop (Chapter 58) or a retry storm. Set it deliberately and review it when traffic grows.

Rate limit headers are internal telemetry. `anthropic-ratelimit-*` values describe your organisation's capacity and current utilisation. Do not forward them to end users or into client-side traces; they tell an attacker how close you are to a ceiling and how hard to push.

429 handling is an availability control. A client that retries a 429 without bound converts a temporary throttle into a sustained outage, and manufactures the sharp usage increase that acceleration limits exist to catch. Bounded retries with jitter are a security property as much as a reliability one.

19.37 Real Production Examples

A. The migration that halved the headroom. A team on Sonnet 4.6 adds Sonnet 5 for new traffic and reports that their aggregate 429 rate went *down* more than expected. It had: Sonnet 5 is a separate rate limit group, so the new traffic drew on an empty bucket while the old traffic kept the full 4.x one. The same team, migrating Opus 4.6 to Opus 4.8 a month later, saw no improvement at all — same group.

B. The dashboard that went flat. A caching rollout appeared to cut input tokens by 89 per cent overnight, which nobody believed and everybody celebrated. The dashboard plotted `usage.input_tokens`, which counts only what follows the last cache breakpoint. Total input was unchanged; the *billed* figure fell by rather less, and the *ITPM* figure fell by rather more. Three numbers, one graph, no reconciliation.

C. The eleventh of the month. A Build-tier organisation on a pilot found all API traffic failing with no 429s. They had reached the $1,000 monthly cap on day eleven. The engineering response — retries, a circuit breaker, a model downgrade — could not have worked, because none of them were the constraint. The fix was a tier change.

D. The retry decorator that multiplied. A service wrapped every SDK call in a five-attempt retry decorator. The SDK retries twice by default. The effective attempt count was fifteen, and during a brief 529 the service generated enough traffic to trip acceleration limits, converting a two-minute upstream blip into a twenty-minute self-inflicted outage.

19.38 Certification Notes

Concept	Exam domain	Why it is examinable
Rate limit versus spend limit	D5 Context Management & Reliability	Two constraints, two failure modes, two remedies.
RPM / ITPM / OTPM as independent buckets	D5	The core mechanical fact of the limit surface.
Cache-aware ITPM	D5	The platform's most distinctive limit behaviour.
`max_tokens` does not affect OTPM	D5	A widely held false belief.
Token bucket and burst behaviour	D5	Explains the load-test paradox.
`retry-after` and doomed early retries	D5	The documented contract behind backoff.

Concept	Exam domain	Why it is examinable
Acceleration limits	D1 Agentic Architecture & Orchestration	Why cutovers ramp.
Response header families	D5	Pre-emptive control versus reactive retry.
Usage tiers and automatic placement	D5	Tiers are earned, not purchased.
Rate limit groups spanning model IDs	D1	Migration planning.
Org / workspace / API key hierarchy	D5	Keys do not carry limits.
Batch limits and the queue depth	D2 Tool Design & MCP Integration	A separate bucket and a separate unit.
Independent `count_tokens` and Files limits	D5	Design freedom most people miss.
Service tiers versus usage tiers	D5	A deliberate naming collision.
Usage and Cost API attribution	D5	Chargeback and reconciliation.

19.39 Twenty Practice Questions

1. Name the two kinds of limit and say what happens when you reach each.
2. What are the three rate limit buckets for the Messages API?
3. Which `usage` fields count toward ITPM, and which does not?
4. Which single model counts cache reads toward ITPM?
5. Does raising `max_tokens` reduce your available OTPM?
6. What algorithm does the API use for rate limiting, and what follows from it for a load test?
7. What is the monthly spend cap on each of Start, Build and Scale?
8. Where are rate limits set: per key, per workspace, or per organisation?
9. Can the default workspace have a rate limit override?
10. Which four Opus models share one rate limit group?
11. What exactly does `retry-after` promise about earlier retries?
12. What is an acceleration limit and what triggers it?
13. What do the undifferentiated `anthropic-ratelimit-tokens-*` headers report?
14. To what precision are token `-remaining` headers reported?
15. What are the Batch API's two distinct request limits, and what does each count?
16. What is the `count_tokens` RPM at each tier, and does it draw on the Messages limit?
17. Name the three service tiers and say which is no longer purchasable.
18. Does `inference_geo: "us"` give you a separate rate limit pool?

19. What HTTP status is returned for a request over 32 MB on the Messages API?
20. Which endpoint returns every model ID and alias that shares a rate limit group?

19.40 Answers and Explanations

1. **Rate limits** throttle per minute and return a **429** you may retry; **spend limits** cap monthly cost and **pause** usage until the next calendar month. Answering "they both stop your requests" misses that only one is self-healing.
2. **RPM, ITPM and OTPM**, measured independently. All three must have room. "Tokens per minute" as a single figure is the other providers' model, not this one.
3. `input_tokens` and `cache_creation_input_tokens` count; `cache_read_input_tokens` **does not**. Remember also that `input_tokens` is only what follows the last breakpoint.
4. **Claude Haiku 3.5**, marked with a dagger in the published tables. Every current model excludes cache reads.
5. **No.** OTPM is metered on tokens actually produced; `max_tokens` reserves nothing. The documentation says there is no rate limit downside to a higher value.
6. A **token bucket**, replenished continuously. Therefore an idle bucket is full, and the first minute of a load test delivers the burst allowance *plus* a minute's income — 36.1 per cent over the sustainable rate in section 19.28.
7. **$500 / $1,000 / $200,000.** Custom has none. The factor-of-200 jump from Build to Scale is the one to remember.
8. **Per organisation.** Workspaces may be set *lower*; API keys carry no limits at all.
9. **No.** Overrides require a named workspace, which is a reason to keep production traffic out of the default one.
10. **Opus 4.8, 4.7, 4.6 and 4.5.** Opus 5 has its own; Sonnet 4.6 and 4.5 share a separate one; Sonnet 5 has its own.
11. That it is the number of seconds to wait, and that **earlier retries will fail**. It is a contract, not a suggestion.
12. A 429 caused by a **sharp increase in usage** even within your published ceilings. Ramp traffic gradually; a retry storm can trigger it by itself.
13. The **most restrictive limit currently in effect** — a workspace limit if one applies, otherwise the sum of input and output remaining. Not a fourth bucket.
14. **Rounded to the nearest thousand.** Treat as an upper bound and keep a margin.
15. A **requests-per-minute** limit on the Batch endpoints, and a **maximum number of batch requests in the processing queue** (200,000 / 300,000 / 500,000). Separately, a single batch holds at most 100,000 requests or 256 MB.
16. **2,000 / 4,000 / 8,000 RPM.** It is free and **independent**: usage of one does not count against the other.
17. **Standard, Priority and Batch. Priority capacity commitments are no longer available for purchase**; existing commitments run to their contract end.

18. **No.** `us` and `global` currently draw from the same pool. It costs 1.1× on price and on Priority burn-down.
19. **413** `request_too_large` — returned by Cloudflare on the direct API before the request reaches Anthropic.
20. `GET /v1/organizations/rate_limits`, whose `model_group` entries carry a `models` list of every ID and alias in the group.

19.41 Ten Practical Exercises

1. Add a `web_search` and a `files` group to `limits.py`, and make `group_for` handle non-model group types.
2. Extend `Gateway` to model acceleration limits: refuse when the trailing ten-second rate exceeds twice the trailing sixty-second rate.
3. Add a fourth strategy that honours `retry-after` exactly with no jitter, and measure the thundering herd it creates with 120 workers.
4. Write `explain(workload, tier)` that prints one sentence naming the binding bucket and the factor by which it binds.
5. Extend `planner.py` to accept a mixed fleet — part Haiku 4.5, part Sonnet 5 — and plan each group separately.
6. Add a Batch path to the planner: given a deferrable fraction, output synchronous demand, batch queue depth and total spend.
7. Model workspace limits: two workspaces over-subscribing one organisation bucket, and measure the starvation.
8. Add a `spend_alarm(workload, day_of_month)` that projects month-end spend and returns the day the cap is reached.
9. Make the pacer shared across four simulated processes and quantify the over-subscription when each keeps its own mirror.
10. Replace the ITPM estimate with a `count_tokens` call charged against its own bucket, and confirm the bucket is never the binding one.

19.42 Five Challenge Labs

1. **The optimal cache breakpoint.** Given a prompt with three candidate breakpoints and a hit-rate model for each, find the placement that maximises ITPM headroom — and show that it is not always the one that minimises cost.
2. **A fair-share scheduler.** Implement weighted fair queuing across five tenants sharing one ITPM bucket, and show that no tenant can starve the others regardless of its offered load.
3. **The migration planner.** Model a phased cutover from Sonnet 4.6 to Sonnet 5 across two rate limit groups, subject to acceleration limits, and find the fastest safe ramp.
4. **Reconciliation.** Generate a month of synthetic usage, aggregate it as the Usage API would at `1d`, and reconcile it against a cost report — including the Priority Tier rows that the cost endpoint omits.

5. **The capacity request.** Produce, automatically from a measured workload, the exact artefact the documentation asks for: models, peak input and output tokens per minute per model, and the cached share of input.

19.43 Ten Interview Questions

1. A service is at 8 per cent of RPM and getting 429s. Walk me through your diagnosis.
2. Why does prompt caching increase throughput and not merely reduce cost?
3. A colleague proposes issuing separate API keys per team to isolate their rate limits. What do you say?
4. Traffic stopped and there are no 429s. What do you check, in what order?
5. Explain why a sixty-second load test can pass and production fail ninety seconds later.
6. Under what circumstances does immediate retrying reduce throughput, and under what circumstances does it not?
7. What is the difference between a usage tier and a service tier?
8. You are migrating Opus 4.6 to Opus 4.8 for capacity reasons. What will happen?
9. How would you decide between requesting a rate limit increase and moving work to Batch?
10. Design the alerting for a system that must never be surprised by either kind of limit.

19.44 Summary

Rate limits look like one number and are not. They are a small family of independent constraints, measured in different units, over different timescales, failing in different ways — and the discipline this chapter asks for is simply to keep them apart. **Rate limits** throttle a minute and return a 429 you may retry. A **spend cap** stops a month and returns nothing at all, because usage pauses until the calendar releases it. Both are set at the organisation level; neither is a property of an API key. A team that has collapsed the two will, sooner or later, spend an afternoon debugging the model layer for an incident that lives on the billing page.

Within the rate limits there are three buckets, not one. Requests per minute counts calls. Input tokens per minute counts uncached input and cache writes, and **not cache reads** — which is the platform's most distinctive limit behaviour and the one that most changes designs, because it makes a cache breakpoint into a rate limit increase you can deploy yourself. Output tokens per minute counts tokens **as they are produced**, so `max_tokens` reserves nothing and the widespread habit of trimming it to protect throughput protects nothing. The three are independent: headroom in two of them does not help you in the third, and for any retrieval-heavy, long-context or agentic workload the third is ITPM. The chapter's planner puts a number on how lopsided that can be — 83.3 per cent of RPM, 83.3 per cent of OTPM and **750.0 per cent of ITPM** on the same traffic, on a workload whose input-to-output ratio is 45:1 against a published bucket ratio of 5:1.

The mechanism underneath is a token bucket, replenished continuously, and it has one consequence that catches almost everybody: an idle bucket is a full bucket, so the first minute of load spends the standing balance as well as the minute's income. Across three client strategies the first minute delivered **756** requests against a sustainable ceiling of **555.6** — an overstatement of **36.1 per cent** — and the last minute of a ten-minute run landed on the ceiling to within half a per cent. A load test that reports a mean,

or that runs for sixty seconds, is reporting the burst allowance and calling it capacity.

What a client does about all this turned out to be less dramatic and more decisive than the folklore suggests. Retrying the instant a 429 arrives is guaranteed to fail — the documentation states that earlier retries will fail, and 36,983 of the naive client's 38,879 refusals arrived before a deadline the server had already published. But the honest measurement is that this cost **3.2 per cent** of completed work, not half of it, because the bucket refills at the same rate whatever the client does; the loss came from doomed requests holding connections in a bounded pool, and with an unbounded pool it would have been zero. The real indictment of the naive client is not that it is slow. It is that it generates **7.08×** the traffic to do the same work, and manufactures precisely the sharp usage increase that acceleration limits exist to catch. Exponential backoff with jitter — what the SDKs already do — halves the waste for four lines. An admission controller that reserves locally before sending and syncs downward from the response headers takes **zero** 429s across ten minutes of deliberately over-subscribed load, at every burst granularity, without ever being told what the granularity is.

That last point is where Part 3 ends and where it has been heading since Chapter 14. The model registry established that a fact is verified on a date or it is UNKNOWN. Chapter 15 made the unit of choice a (model, effort) pair. Chapter 16 established that the conversation is a list you resend, Chapter 17 that the client owns the message the server merely streams, and Chapter 18 that the configuration is part of the prompt and therefore part of the cache. This chapter adds the constraint that makes all of them operational: **the platform tells you where you stand on every single response, and a system that reads it does not need to guess.** Part 4 turns from what the API will accept to what you should put in it.

KEY TAKEAWAYS

1. **Two kinds of limit.** A rate limit throttles a minute and self-heals; a spend cap pauses a month and does not. Both live at the organisation level.
2. **Three independent buckets** — RPM, ITPM, OTPM. Any one of them refuses; headroom in the others does not help.
3. **Cache reads do not draw ITPM** on any current model. A cache breakpoint is a rate limit increase that needs nobody's approval — worth **5.00×** on this chapter's workload.
4. `max_tokens` **reserves no OTPM.** Output is metered as produced. Trim it for latency, never for throughput.
5. **One request has three token counts** — 18,000 in the context window, 3,600 against the bucket, 5,440 on the invoice — and they move independently.
6. **An idle bucket is a full bucket.** The first minute overstated the sustainable rate by **36.1 per cent**. Report the last minute.
7. **An early retry is refused by contract**, and a retry storm is a self-inflicted acceleration limit. Naive retrying cost 3.2 per cent of throughput and **7.08×** the traffic.
8. **Limits are per organisation and per rate limit group.** More API keys buy nothing; Opus 4.8 through 4.5 share one bucket.
9. **Batch,** `count_tokens`**, Files and Managed Agents have separate buckets.** Free, independent capacity you are probably not using.
10. **Plan the rate limit and the spend cap separately and take the higher tier.** Here the rate limits said Build and the $1,000 cap said Scale.

19.45 Glossary

Term	Definition
Rate limit	A per-minute ceiling on requests or tokens. Exceeding it returns 429; capacity self-heals.
Spend limit / spend cap	A monthly USD ceiling. Reaching it **pauses** API usage until the next calendar month.
Usage tier	Start, Build, Scale or Custom. Assigned automatically from usage history; carries limits and a cap.
Service tier	How a request is scheduled: Standard, Priority or Batch. A different axis from usage tier.
RPM	Requests per minute. One per accepted request, whatever its size.
ITPM	Input tokens per minute. Uncached input plus cache writes; not cache reads.
OTPM	Output tokens per minute, metered as tokens are produced.
Token bucket	The rate limiting algorithm: capacity replenished continuously, not reset on a boundary.
Burst allowance	How much capacity may accumulate in an idle bucket. Not published; may be enforced at second scale.
Cache-aware ITPM	The rule that `cache_read_input_tokens` do not draw ITPM, except on Claude Haiku 3.5.
Rate limit group	The unit limits are enforced against. May span several model IDs and aliases.
Acceleration limit	A 429 triggered by a sharp increase in usage, even within published ceilings.
`retry-after`	Seconds to wait after a 429. Retries sent earlier are documented to fail.
429 `rate_limit_error`	You exceeded a limit. Distinct from 529 `overloaded_error`, which is platform-wide.
413 `request_too_large`	Over the byte ceiling: 32 MB Messages, 256 MB Batch, 500 MB Files.
Admission control	Deciding client-side not to send a request that would be refused.
Workspace	An organisational unit that may carry rate and spend limits **lower** than the organisation's.
Admin API key	`sk-ant-admin...`. Required for the Rate Limits, Usage and Cost APIs. Not an application credential.
Enqueued batch requests	Individual batched requests awaiting the model. A separate limit from Batch RPM.
Priority Tier	Prioritised scheduling under a capacity commitment. No longer available for purchase.

19.46 Further Reading

Primary sources, at `platform.claude.com/docs/en/`, verified **30 July 2026**:

- `api/rate-limits` — the tier tables, cache-aware ITPM, batch limits, response headers, workspace limits.

- `api/errors` — the full status code list, request size ceilings, acceleration limits and SDK retry behaviour.
- `api/service-tiers` — Standard, Priority and Batch, burn-down rates and the `service_tier` parameter.
- `build-with-claude/batch-processing` — batch limits, the 24-hour window, pricing and the agentic loop inside a batch.
- `build-with-claude/token-counting` — the independent, free RPM ladder.
- `manage-claude/rate-limits-api` — reading your own configured limits, per organisation and per workspace.
- `manage-claude/usage-cost-api` — attribution, grouping, the five-minute freshness and the Priority Tier exclusion.

Where to go next in this book. Chapter 20 opens Part 4 with the principles of prompting — where a prompt's *shape* becomes the thing that determines the ITPM draw this chapter has been measuring. **Chapter 29** develops errors, retries, timeouts and backoff into a complete client policy, of which section 19.27's admission controller is one component. **Chapter 30** is the proper treatment of batch processing and prompt caching, including the 1-hour duration and cache-hint routing this chapter uses without pricing. **Chapter 57** builds the observability that turns `anthropic-ratelimit-*` headers into alerts. **Chapter 60** returns to all of this at scale, where quota becomes an architectural input rather than an operational surprise.

Looking back across Part 3. Six chapters have built one thing: a defensible account of what the platform is. Chapter 14 gave the models and the rule that an unverified fact must raise; Chapter 15 gave the method for choosing among them and the price of a small evaluation set; Chapter 16 gave the wire; Chapter 17 gave streaming, vision and documents; Chapter 18 gave thinking as a configuration with a protocol and a cache identity; and Chapter 19 gave the limits, quotas and attribution that decide whether any of it survives contact with volume. What none of them have addressed is the content of the request. That is Part 4.